Vietnam National University - Ho Chi Minh City
University of Science
Faculty of Information Technology

TEST PLAN

SOFTWARE TESTING PROJECT PLAN

Team Members

Name	ID	Email
Cao Uyển Nhi	22127310	cunhi22@clc.fitus.edu.vn
Lưu Thanh Thuý	22127410	ltthuy22@clc.fitus.edu.vn
Nguyễn Phước Minh Trí	22127424	npmtri22@clc.fitus.edu.vn
Võ Lê Việt Tú	22127435	vlvtu22@clc.fitus.edu.vn
Trần Thị Cát Tường	22127444	ttctuong22@clc.fitus.edu.vn

Supervisors

Name
Teacher Trần Duy Hoàng
Teacher Hồ Tuấn Thanh
Teacher Trương Phước Lộc

Aug 10, 2025

Contents

Revision History

Version	Date	Author	Description
0.0	2025-08-20	Cao Uyển Nhi	Created initial Test Plan template and document structure
0.1	2025-08-21	Trần Thị Cát Tường	Added introduction, scope, and evaluation mission sections
0.2	2025-08-21	Lưu Thanh Thuý	Drafted outline of planned tests and test approach
0.3	2025-08-22	Nguyễn Phước Minh Trí	Added entry/exit criteria and test workflow sections
0.4	2025-08-22	Võ Lê Việt Tú	Contributed environmental needs and staffing/training details
0.5	2025-08-23	Cao Uyển Nhi	Expanded deliverables section and smoke test suite content
0.6	2025-08-23	Trần Thị Cát Tường	Updated milestones and refined risks/dependencies section
0.7	2025-08-24	Lưu Thanh Thuý	Completed traceability strategies and approval/signoff details
0.8	2025-08-24	Cao Uyển Nhi	Consolidated all sections and reviewed formatting for consistency
1.0	2025-08-25	Cao Uyển Nhi	Finalized Test Plan for HW#08 submission

Chapter 1

Introduction

1.1 Purpose

The purpose of this Test Plan is to define the strategy and scope of testing The Toolshop application (Sprint 5 with known issues). It ensures a structured approach to verifying functionality, performance, and quality, aligning with the requirements of the **CSC13003 – Software Testing course**.
Key objectives:

- Verify that core e-commerce workflows (browse, add to cart, checkout, payment, invoice) function correctly.
- Validate backend APIs for correctness, error handling, and robustness.
- Evaluate GUI usability and adherence to validation rules.
- Identify and document defects with severity and priority levels.
- Provide measurable insights on quality risks and coverage.
- Deliver professional test documentation as required by CS423 course outcomes.

1.2 Scope

In-Scope

- Functional testing of authentication (sign-up, sign-in), catalog browsing, cart management, order processing, and payment workflows.
- GUI validation (input forms, navigation menus, layout consistency, responsiveness on supported browsers like Chrome and Firefox).
- API testing (`/users/login`, `/payment/check`, `/invoices`) to ensure data integrity and API contract compliance.
- Automation testing (Selenium for UI, Postman/Newman for API) to enhance efficiency and repeatability.
- Performance testing (load, stress, spike) with JMeter to assess scalability and stability.
- Bug reporting, regression testing after fixes, and a final quality assessment report.

Out of Scope

- Security penetration testing beyond basic SQL injection and XSS validation checks.
- Localization testing (focused on English only, no multi-language support testing).
- Mobile application or native app testing (web-based application only).
- Production deployment testing (no live server or CI/CD pipeline validation).

1.3 Intended Audience

- QA Team (students responsible for execution and documentation)
- Lecturers/TAs (evaluators and mentors)
- Developers (recipients of bug reports and test insights)

1.4 Document Terminology and Acronyms

- SUT: System Under Test (The Toolshop)
- GUI: Graphical User Interface
- API: Application Programming Interface
- JMX: JMeter test script
- CI/CD: Continuous Integration / Deployment

1.5 References

- Project Repo: Practice Software Testing – Sprint5-with-bugs
- CS423 – CSC13003 assignment description

1.6 Document Structure

This Test Plan follows the Rational Unified Process structure: defining mission, scope, approach, deliverables, workflow, and risks.

Chapter 2

Evaluation Mission and Test Motivation

2.1 Background

The Toolshop is an e-commerce web application designed to facilitate user management, product catalog browsing, shopping cart operations, and checkout processes. Sprint 5 of the application has been intentionally embedded with known bugs to serve as a practical testing ground for students, aligning with the CSC13003 – Software Testing course at the Faculty of Information Technology, University of Science, Ho Chi Minh City. The application, hosted in the `/sprint5-with-bugs` folder of the repository https://github.com/testsmith-io/practice-software-testing/, provides a controlled environment to simulate real-world software defects and test scenarios.

2.2 Evaluation Mission

The primary mission of this testing initiative is to systematically uncover functional and performance defects within *The Toolshop* application. This includes validating critical e-commerce workflows such as user authentication, product browsing, cart management, checkout, payment processing, and invoice generation. The effort aims to produce comprehensive, professional QA deliverables, including test plans, detailed test cases, bug reports, and test summaries, as mandated by the course learning outcomes. The testing process will also assess GUI usability, API reliability, and system performance under various conditions, ensuring a thorough evaluation that supports defect identification, documentation, and management skills development.

2.3 Test Motivators

- **Known Bugs in Sprint 5**: The intentional inclusion of defects provides a realistic challenge to identify, document, and prioritize issues, enhancing practical testing experience.
- **Academic Requirement**: The course mandates proficiency in multiple testing types, including domain testing, GUI testing, scenario-based testing, automation testing, performance testing, and API testing, which are essential for meeting the assignment objectives (HW#02 to HW#07).
- **E-commerce Workflow Reliability**: Ensuring the stability and correctness of core e-commerce functionalities is critical, as these directly impact user experience and business success, making them a focal point for testing efforts.

- **Professional Skill Development**: The need to create standardized test documentation (e.g., test plans, bug reports) and collaborate effectively in a group setting (HW#08) drives the motivation to align with industry practices and academic expectations.

Chapter 3

Target Test Items

Frontend (Angular UI)

- Login: Verifying user authentication, including valid/invalid credentials, session management, and logout functionality.
- Catalog: Testing product listing, search filters (e.g., price, category), pagination, and detailed product views.
- Checkout: Validating the checkout process, including cart review, shipping details, payment integration, and order confirmation.
- Admin: Assessing admin-specific features like user management, order oversight, and category maintenance.

Backend (Laravel APIs)

- Authentication: Ensuring secure login/logout endpoints, token generation, and handling of expired or invalid tokens.
- User Management: Testing CRUD operations (create, read, update, delete) for user profiles, role assignments, and data consistency.
- Payment: Verifying payment processing endpoints, transaction status checks, and error handling for failed payments.
- Invoice: Validating invoice generation, retrieval, and formatting, including edge cases like invalid order IDs.

Modules

- User Login: Confirming seamless login across devices, including password recovery and multi-factor authentication if applicable.
- Add/Edit User: Testing user profile creation, modification (e.g., name, email), and deletion, with validation for duplicates and data integrity.
- Product Browsing: Ensuring accurate product display, category navigation, and search functionality with real-time updates.
- Cart: Verifying item addition/removal, quantity adjustments, subtotal calculations, and persistence across sessions.
- Payment: Checking payment initiation, validation of payment methods, and successful transaction completion.
- Invoice: Testing invoice generation post-payment, email delivery, and PDF export functionality.

Chapter 4

Outline of Planned Tests

4.1 Test Inclusions

This section details the core testing activities that will be conducted as part of the project. These tests align with the learning outcomes specified in the course, focusing on various testing types such as domain testing, scenario-based testing, GUI testing, automation testing, performance testing, and API testing. The tests will be applied to key features of **"The Toolshop"** application (from the `/sprint5-with-bugs` folder in the repository), ensuring comprehensive coverage of functional and non-functional aspects.
Functional Testing: This will cover essential user workflows and business logic to verify that the application behaves as expected under normal conditions. Specific areas include:

- **Authentication:**
  - **Sign Up:** Verifying new user registration, including form validation, email uniqueness checks, password requirements, and successful account creation.
  - **Sign In:** Testing login with valid/invalid credentials, session establishment, role-based access, and logout functionality.
- **Profile Management:** Ensuring users can update personal details (e.g., name, address, email) and change passwords, with validation for old/new password matching and strength checks.
- **Product Browsing and Catalog:**
  - **Catalog:** Checking product listing, search functionality, filtering by attributes (e.g., price, brand), pagination, and detailed product views.
  - **Categories:** Verifying category navigation, sub-categories, product assignment to categories, and dynamic updates.
- **Shopping Cart and Checkout:**
  - **Cart Management:** Testing adding/removing/updating items, quantity adjustments, persistence across sessions, and total calculations including discounts or taxes.
  - **Checkout:** Validating the end-to-end order placement process, including shipping details, payment simulation, invoice generation, order confirmation, and handling of incomplete checkouts.
- **Contact:** Ensuring submission of inquiries, validation of input fields (e.g., email, message), and confirmation of receipt (e.g., via email or on-screen message).
- **Admin Features:**
  - **Category Management:** Testing creation, editing, deletion of categories, assignment of products, and hierarchy management for admins.

  - **User Management:** Verifying admin abilities to view, edit, delete user accounts, manage roles, and handle user-related data.
  - **Order Management:** Checking order viewing, status updates (e.g., processing, shipped, canceled), refund processing, and integration with invoices.

GUI Testing: Focused on the user interface to ensure usability, consistency, and responsiveness. Key elements to test:

- **Navigation:** Checking menu structures, links, breadcrumbs, and page transitions for intuitive flow and absence of dead ends across all pages, including admin dashboards.
- **Input validation:** Testing form fields for proper error messages, data type enforcement (e.g., email formats, numeric limits), and prevention of invalid submissions in features like sign up, profile updates, and contact forms.

API Testing: Targeting backend endpoints to validate data exchange, security, and reliability. Specific APIs under test:

- `/users/login`: Verifying authentication responses, token generation, and handling of edge cases like expired sessions or invalid credentials.
- `/payment/check`: Ensuring payment status checks, validation of transaction details, and integration with simulated payment gateways.
- `/invoices`: Testing invoice retrieval, generation, and formatting, including error scenarios for invalid invoice IDs.

Automation Testing: Implementing scripted tests for repeatability and efficiency. Tools and scopes:

- **Selenium for UI automation:** Automating browser interactions for end-to-end scenarios like login-to-checkout flows, including cross-browser testing on Chrome and Firefox.
- **Newman for API automation:** Running Postman collections to automate API calls, assertions on response codes, payloads, and performance metrics.

Performance Testing: Assessing the application's scalability and stability under varying loads. Using JMeter for:

- **Load testing:** Simulating multiple concurrent users to measure response times and throughput during peak usage, such as simultaneous checkouts or catalog browsing.
- **Spike testing:** Evaluating system behavior under sudden traffic surges, such as rapid increases in checkout requests or admin order updates.
- **Stress testing:** Pushing the application beyond normal limits to identify breaking points, resource leaks, and recovery mechanisms.

4.2 Other Candidates for Potential Inclusion

These are additional testing areas that may be incorporated if time and resources permit, or if initial testing reveals related issues. They extend beyond the core requirements but could enhance overall quality:

- **Cross-browser Testing:** Extending GUI and functional tests to less common browsers like Safari, Chrome and Firefox to ensure compatibility, focusing on rendering differences, JavaScript execution, and CSS inconsistencies.
- **Accessibility Validation:** Using tools like WAVE or axe to check compliance with WCAG standards, including screen reader compatibility, keyboard navigation, color contrast, and alt text for images.

4.3 Test Exclusions

To maintain focus on the project's scope and deadlines, the following areas will not be tested. These exclusions are based on the assignment guidelines, which emphasize specific testing types without requiring advanced security or platform-specific validations:

- **Security Penetration Testing:** No in-depth vulnerability scanning, ethical hacking, or tests for SQL injection, XSS, or other exploits, as this falls outside the course's core testing types.
- **Mobile App Testing:** The application is web-based; no testing on mobile devices, responsive designs beyond basic GUI checks, or native app features.
- **Production Deployment Testing:** No verification of live server configurations, CI/CD pipelines, or real-world hosting environments, as the focus is on the development sprint with known bugs.

Chapter 5

Test Approach

This section outlines the methodologies and techniques to be employed in testing *The Toolshop* application during the CSC13003 – Software Testing course. The approach is tailored to meet the learning outcomes and assignment requirements, ensuring comprehensive coverage of functional, performance, and quality aspects.

5.1 Initial Test-Idea Catalogs and other reference sources

The initial test ideas are derived from the project description, the *Test Plan Template*, and the CS423 course guidelines. Additional reference sources include the *Practice Software Testing* repository and the Sprint 5 with known issues folder. These resources guide the identification of test scenarios and techniques.

5.2 Testing Techniques and Types

5.2.1 Data and Database Integrity Testing

Aspect	Description
Objective	Ensure data accuracy, consistency, and integrity across the database.
Techniques	Validate data input/output, check for duplicates, and verify referential integrity.
Tools	SQL queries, database management tools.
Focus Areas	User data, product catalog, order history, payment records.

5.2.2 Function Testing

Aspect	Description
Objective	Verify that individual functions (e.g., login, checkout) perform as expected.
Techniques	Black-box testing, equivalence partitioning, boundary value analysis.
Tools	Manual testing, Selenium for automation.
Focus Areas	Authentication, cart management, payment processing, invoice generation.

5.2.3 Business Cycle Testing

Aspect	Description
Objective	Validate end-to-end business processes (e.g., order-to-delivery).
Techniques	Scenario-based testing, workflow analysis.
Tools	Postman for API workflows, manual validation.
Focus Areas	User registration to order completion, payment reconciliation.

5.2.4 User Interface Testing

Aspect	Description
Objective	Ensure the Angular UI is user-friendly, responsive, and visually consistent.
Techniques	GUI testing, cross-browser testing (Chrome, Firefox).
Tools	Selenium, manual inspection tools (e.g., browser developer tools).
Focus Areas	Navigation, input validation, layout consistency.

5.2.5 Performance Profiling

Aspect	Description
Objective	Assess the application's performance under normal conditions.
Techniques	Benchmarking, response time measurement.
Tools	JMeter, browser performance tools.
Focus Areas	Page load times, API response times, concurrent user actions.

5.2.6 Load Testing

Aspect	Description
Objective	Evaluate system behavior under expected user loads.
Techniques	Simulated user load testing.
Tools	JMeter, LoadRunner.
Focus Areas	Checkout process with 100 concurrent users, catalog browsing.

5.2.7 Stress Testing

Aspect	Description
Objective	Determine the system's breaking point under extreme conditions.
Techniques	Overload testing, failure injection.
Tools	JMeter, custom scripts.
Focus Areas	Server response under 500+ concurrent users, payment failures.

5.2.8 Volume Testing

Aspect	Description
Objective	Test system performance with large data volumes.
Techniques	Data volume scaling, stress on database.
Tools	JMeter, database load simulators.
Focus Areas	10,000 product catalog, 1,000 simultaneous orders.

5.2.9 Security and Access Control Testing

Aspect	Description
Objective	Ensure secure access and protect against basic vulnerabilities.
Techniques	Role-based testing, basic SQLi/XSS checks.
Tools	OWASP ZAP, manual security testing.
Focus Areas	User authentication, admin access, payment data protection.

Chapter 6

Entry and Exit Criteria

This section defines the conditions for initiating and concluding the testing phases of *The Toolshop* application. These criteria ensure that testing aligns with the project timeline and quality standards, reflecting the group assignment requirements.

6.1 Test Plan

6.1.1 Test Plan Entry Criteria

Criteria	Description
Test Environment Ready	Test platform (e.g., local setup with Chrome/Firefox) is fully configured and accessible.
Requirements Defined	All functional and non-functional requirements from the Sprint 5 description are documented.
Test Cases Prepared	Test cases for HW#02 to HW#07 (Domain, GUI, Automation, Performance, API) are drafted and reviewed.
Team Coordination	Task distribution among group members is agreed upon and documented for HW#08.

6.1.2 Test Plan Exit Criteria

Criteria	Description
Test Coverage Achieved	Minimum 90% coverage of identified test cases across all testing types.
Defects Resolved	Critical and major bugs (as per bug reports) are addressed or documented for deferral.
Documentation Complete	Final report (HW#08) including Test Plan, Test Cases, Bug Reports, and Test Report is submitted.
Approval Received	Review and approval from lecturers/TAs via Moodle.

6.1.3 Suspension and Resumption Criteria

Criteria	Description
Suspension	Testing halts if critical system failures (e.g., server downtime) exceed 48 hours or if 50% of test cases fail repeatedly.

Criteria	Description
Resumption	Resumes once the environment is restored, and a root cause analysis is documented, approved by the TA.

6.2 Test Cycles

6.2.1 Test Cycle Entry Criteria

Criteria	Description
Build Availability	The Sprint 5 with known issues build is available in the repository.
Environment Setup	Test environment (e.g., Chrome/Firefox browsers, local server) is configured and validated by 07, August 25, 2025.
Test Cases Ready	Test cases for the current cycle (e.g., HW#02 Domain Testing) are reviewed and approved by the group and TAs.
Resource Allocation	All team members (as per group assignment) are assigned tasks, and tools (e.g., Selenium, JMeter) are accessible.

6.2.2 Test Cycle Exit Criteria

Criteria	Description
Test Completion	All test cases for the cycle (e.g., HW#02) are executed, with results documented by the cycle deadline.
Defect Reporting	All identified bugs are logged with severity/priority levels and reproduction steps in the HW#08 Bug Reports.
Coverage Achieved	Minimum 85% test coverage for the cycle's focus area (e.g., domain testing) is met.
TA Review	Preliminary results are submitted to TAs for feedback before cycle closure.

6.2.3 Test Cycle Abnormal Termination

Criteria	Description
Critical Failure	Testing is suspended if a critical system failure (e.g., server crash) persists for over 24 hours.
Significant Defects	Cycle ends prematurely if 60% of test cases fail with major defects, requiring a build alteration.
Resource Unavailability	Termination occurs if key tools (e.g., JMeter) or team members are unavailable for 48+ hours.
Build Alteration	The intended build candidate is altered if new bugs render the current version untestable, pending TA approval.

Chapter 7

Deliverables

7.1 Test Evaluation Summaries

The test evaluation summaries for *The Toolshop* application (Sprint 5 with known issues) will consist of concise reports detailing the outcomes of each test cycle, covering assignments HW#02 (Domain Testing) through HW#07 (API Testing). The content will include pass/fail rates, key functional and performance findings, and a qualitative assessment of test execution.

7.2 Reporting on Test Coverage

Reports on test coverage will measure the extent of testing across all required types (Domain, GUI, Automation, Performance, API) for *The Toolshop*, as outlined in the project description. The content will include a detailed breakdown of tested features (e.g., login, catalog, checkout), the number of test cases executed versus planned, and a coverage percentage (targeting a minimum of 85%), presented in a narrative document with supporting charts.
These reports will be generated after the completion of each HW assignment cycle (e.g., post-HW#02, HW#03) and consolidated into the HW#08 Final Report. The extent of testing will be recorded using manual logs, tracked with tools like Selenium for UI automation, JMeter for performance, and Postman/Newman for API tests, with the final report due by the submission deadline.

7.3 Perceived Quality Reports

Perceived quality reports will assess the overall quality of *The Toolshop* based on test outcomes, focusing on usability, performance, and reliability. The content will include a narrative evaluation of user interface responsiveness, API robustness, and system stability under load, supplemented by an analysis of incident logs and change requests against test coverage data.
These reports will be formatted as part of the Test Report section in the HW#08 Final Report, produced after each major testing phase (e.g., post-HW#04 GUI Testing, HW#06 Performance Testing) and finalized before submission. Quality will be measured through manual observations, automated test results from Selenium and JMeter, and feedback from group members, with the final report reviewed by Teacher Tran Duy Hoang.

7.4 Incident Logs and Change Requests

Incident logs and change requests will document all issues encountered during testing of *The Toolshop*, such as functional defects (e.g., failed login attempts) and performance bottlenecks. The method involves recording incidents with severity levels, reproduction steps, and status

in a structured format. Change requests will track suggested fixes or enhancements, linked to specific test cases. These will be managed using a shared spreadsheet or document, integrated into the HW#08 Bug Reports, and tracked with manual updates or a basic issue tracking tool (e.g., Excel or Google Sheets).

7.5 Smoke Test Suite and Supporting Test Scripts

The smoke test suite will consist of a set of basic tests to verify the core functionalities of *The Toolshop*, including login, catalog browsing, and checkout processes, ensuring no critical regressions in subsequent Sprint 5 builds. Supporting test scripts will be developed using Selenium for UI testing and Newman for API testing (e.g., `/users/login`), designed to run quickly and confirm system stability. These assets will be documented and delivered as part of the HW#05 Automation Testing submission, with scripts maintained in the group repository, reviewed by Teacher Ho Tuan Thanh, and updated as needed until the final HW#08 submission.

7.6 Additional Work Products

7.6.1 Detailed Test Results

Detailed test results will comprise a collection of spreadsheets or a repository documenting the step-by-step outcomes of each test case executed during HW#02 to HW#07. These will include decisions and comments, maintained manually or via a test management tool if available. While useful for reference, they are optional deliverables and not the primary measure of success, to be included in the HW#08 Final Report as supporting data, reviewed by the group leader.

7.6.2 Additional Automated Functional Test Scripts

Additional automated functional test scripts will include optional enhancements beyond HW#05, such as extra Selenium scripts for edge cases (e.g., payment failures) or Newman scripts for API. These will be stored as source code files or in a repository, offering flexibility for future testing but not used to assess the Test Plan's success. They will be contributed voluntarily by the automation team and submitted with HW#08 if completed, pending TA approval.

7.6.3 Test Guidelines

Test guidelines will provide a comprehensive set of instructions for the *The Toolshop* testing effort, including test-idea catalogs (e.g., scenarios for HW#03), good practice guidance (e.g., bug reporting standards), test patterns (e.g., boundary testing), fault and failure models (e.g., server crashes), and automation design standards (e.g., Selenium best practices). These will be documented as a separate section in the HW#08 Test Plan, serving as a reference but not a success metric, drafted by the group.

7.6.4 Traceability Matrices

Traceability matrices will map test cases to requirements from the *The Toolshop* specification, ensuring all critical functionalities are tested. These will be created using MS Excel, showing relationships between test inputs, outputs, and verified requirements, and included in the HW#08 Final Report. While valuable for verification, they are an optional deliverable and not the primary measure of the Test Plan's success, maintained by the group leader.

Chapter 8

Testing Workflow

The workflow ensures a structured approach to testing, aligning with the HW#02 to HW#08 assignment requirements and reflecting the current date and time (Monday, August 25, 2025).

1. **Analyze Requirements**

- Conduct a thorough review of the *The Toolshop* project requirements, including the Sprint 5 description from the repository.
- Identify key functionalities (e.g., login, catalog, checkout) and non-functional aspects (e.g., performance, usability) as outlined in HW#02 to HW#07.
- Collaborate with group members to clarify ambiguities with TAs and document findings for the HW#08 Test Plan.

2. **Design Test Cases**

- Develop test cases for each testing type (Domain, GUI, Automation, Performance, API) based on identified requirements.
- Include positive, negative, and edge cases (e.g., invalid login credentials, bulk order processing) with clear inputs, expected outputs, and criteria, as per HW#03 guidelines.
- Review and refine test cases within the group, ensuring coverage of known bugs.

3. **Setup Environment**

- Configure the test environment using local setups with Chrome and Firefox browsers, as specified for *The Toolshop*.
- Install and validate tools (e.g., Selenium, JMeter, Postman) required for HW#05 and HW#06, ensuring compatibility with the Sprint 5 build.
- Verify network stability and server access by August 25, 2025, and document setup details for HW#08 submission.

4. **Execute Manual Tests**

- Perform manual testing of core workflows (e.g., user registration, payment processing) as outlined in HW#02 and HW#04.
- Record step-by-step results (e.g., OK, NOK) for test cases like LOGIN-001 and CHECKOUT-001, noting any deviations or bugs.

5. **Run Automation**

- Execute automated test scripts developed for HW#05, using Selenium for UI (e.g., navigation testing) and Newman for API (e.g., /users/login).

- Validate script functionality against the Sprint 5 build, addressing any failures (e.g., timing issues) with adjustments.
- Schedule automation runs weekly, with results integrated into the HW#08 Final Report by the submission deadline.

6. **Run Performance Tests**

- Conduct performance testing as per HW#06 using JMeter to simulate loads (e.g., 100 concurrent users on checkout) and measure response times.
- Perform stress and volume tests to identify breaking points, focusing on catalog browsing and payment APIs.

7. **Log Defects**

- Record all identified defects (e.g., UI misalignment, API timeouts) with severity (Critical, Major, Minor), reproduction steps, and screenshots in a shared document.
- Assign bug IDs and link them to specific test cases.
- Update logs daily during testing, with a consolidated Bug Report submitted as part of HW#08.

8. **Retest and Regression**

- Retest fixed defects to verify resolutions, scheduled after developer feedback.
- Conduct regression testing using the smoke test suite and additional scripts to ensure no new issues arise, covering all HW#02 to HW#07 scopes.
- Document retest outcomes and regression results, integrating them into the HW#08 Test Case Summary.

9. **Summarize in Reports**

- Compile all testing data into a comprehensive HW#08 Final Report, including Test Plan, Test Cases, Test Case Summary, Bug Reports, and Test Report.
- Include metrics (e.g., 85% test coverage), qualitative assessments, and stakeholder recommendations, reviewed by Teacher Tran Duy Hoang.
- Finalize and submit the report via Moodle on the project deadline, ensuring TA approval.

Chapter 9

Environmental Needs

9.1 Base System Hardware

Component	Specification	Notes
Server (local)	CPU: 4 cores, RAM: 8 GB, Storage: 50 GB	Runs backend application and database
Client machines	CPU: Dual-core 2 GHz, RAM: 4 GB, Disk: 10 GB	Student laptops used for front-end testing
Network	Stable Wi-Fi 50 Mbps+	Required to ensure performance tests with JMeter

9.2 Base Software Elements in the Test Environment

Software Element	Version / Toolchain	Purpose
OS (client)	Windows 10/11, Ubuntu 20.04	Operating systems for testers' machines
Browsers	Chrome 126, Firefox 128, Edge 126	Official cross-browser testing targets
Backend	Laravel 8.x, PHP 8.x	Web application backend framework
Database	MySQL 8.x	Stores product, order, and user data
Containerization	Docker 24.x	Rapid setup for backend and database

9.3 Productivity and Support Tools

Tool	Usage
Postman/Newman	API testing and automated collection execution in CI/CD
Selenium WebDriver	GUI/UI automation testing
JMeter	Performance testing (load, stress, spike)
GitHub Issues	Bug tracking, change requests, and defect management
Google Sheets	Test Case management, Incident Logs, Traceability Matrix
GitHub Actions	CI/CD pipeline for automated test suites

9.4 Test Environment Configurations

Configuration Type	Details
Localhost setup	Backend and database deployed via Docker Compose on student machines
Browser configs	Chrome (Windows/Linux), Firefox (Ubuntu), Edge (Windows)
Minimal config	Client laptop with 2 GB RAM running Chrome to validate basic usability
Average config	Client laptop with 4 GB RAM, stable Chrome/Firefox for main testing
Performance config	JMeter run on 8 GB RAM machine simulating 50–200 virtual users
Accessibility check	Basic screen reader test (NVDA on Windows) – optional

Chapter 10

Responsibilities, Staffing, and Training Needs

10.1 People and Roles

Role	Responsibilities	Assigned
Test Manager	Oversees the entire test effort, approves the Test Plan, manages risks, reviews deliverables, ensures deadlines are met.	Group leader
Test Analysts	Design test cases for functional, scenario-based, GUI, and API testing; ensure coverage of requirements.	All Team
Manual Testers	Execute test cases manually, log defects, retest after fixes, validate usability issues.	All Team
Automation Engineer	Develops and maintains Selenium scripts (UI) and Postman/Newman collections (API); integrates automation into CI/CD pipeline.	All Team
Performance Engineer	Designs and executes JMeter scenarios for load, stress, and spike tests; analyzes performance results.	All Team
Test System Admin	Sets up and maintains test environment (Docker, databases, browsers, configurations).	All Team
Developers (supporting role)	Provide builds, fix defects, clarify requirements.	All Team

10.2 Staffing and Training Needs

Area	Training / Knowledge Needed	Plan / Approach
Selenium Automation	Writing and running WebDriver scripts for UI regression testing	Short internal workshop; practice on login/logout flows
Postman/Newman	Creating collections, writing assertions, integrating into GitHub Actions	Hands-on sessions during HW#07; group peer support
JMeter	Setting up thread groups, simulating load, analyzing reports	Instructor tutorials + online docs; dry run before HW#06 submission
Docker Environment	Running Laravel + MySQL using Docker Compose	One member documents setup guide for the group

Area	Training / Knowledge Needed	Plan / Approach
Defect Management	Using GitHub Issues or spreadsheets for bug tracking	Standardize defect logging format across group
Test Design	Applying Boundary Value Analysis, Equivalence Partitioning, Scenario Testing	Already practiced in HW#02 and HW#03; guidelines shared in Test Plan

Chapter 11

Iteration Milestones

The following milestones outline the key checkpoints for the test effort on *The Toolshop* (Sprint 5 with Bugs). Since the final submission is due on **25-Aug-2025**, milestones have been condensed to align with this strict deadline.

Milestone	Description	Target Date & Time	Owner / Reviewer
Iteration Plan Approved	Test Plan finalized; scope, approach, responsibilities confirmed.	23-Aug-2025	Test Manager + QA Team Lead
Test Cases Designed	Domain, Scenario, GUI, API, and Performance test cases completed.	24-Aug-2025	Test Analysts, reviewed by peers
Test Environment Setup	Docker backend, database seeded, browsers/tools configured.	24-Aug-2025	Test System Admin
Cycle 1 Execution	Manual functional and GUI tests executed; defects logged.	24-Aug-2025	Manual Testers
Cycle 2 Execution	API testing with Postman/Newman + smoke automation scripts.	25-Aug-2025	Automation Engineer
Cycle 3 Execution	JMeter load/stress tests executed, results captured.	25-Aug-2025	Performance Engineer
Regression Cycle	Quick regression and smoke tests on fixed defects.	25-Aug-2025	Entire QA Team
Test Case Summary Ready	Consolidated summary of pass/fail and coverage metrics.	25-Aug-2025	Test Manager + Analysts
Final Test Report Draft	Full Test Report compiled (summaries, bug logs, coverage, conclusions).	25-Aug-2025	QA Team Lead
Final Submission (HW#08)	Full deliverables submitted to Moodle.	25-Aug-2025	Entire QA Team, Instructor review

Chapter 12

Risks, Dependencies, Assumptions, and Constraints

12.1 Risks

Risk	Mitigation Strategy	Contingency (Risk is realized)
Sprint 5 build is unstable or inaccessible	Maintain a backup Docker image and pin a stable commit; run smoke test after each pull	Revert to stable build; run only critical smoke/regression tests; document untested areas
Test data is inadequate or inconsistent	Prepare seed scripts with product and user data; verify DB before execution	Redefine test data; refresh database; rerun impacted test cases
Submission deadline is too strict	Prioritize high-risk features (login, checkout, payment); parallelize team effort	Deliver partial coverage with explanation of gaps; include "not tested" list
Tool misconfiguration (Selenium, JMeter, Newman, Docker)	Share setup guide; validate environments in advance	Fall back to manual execution or alternative machines; attach raw logs/screenshots
Limited hardware resources (student laptops)	Use lightweight browsers; stagger JMeter runs with fewer users	Scale down load tests; report relative performance trends instead of absolute metrics

12.2 Dependencies

Dependency	Potential Impact	Owners
Availability of Sprint 5 build on GitHub	Without build, no functional or regression tests can run	Development team / QA setup
Seeded database with test data	Test cases for checkout and invoices cannot execute without sample data	Test System Admin
Access to required tools (Postman, JMeter, Selenium, Docker)	Missing or misconfigured tools block test execution	Each assigned tester
Moodle submission portal availability	Final deliverables cannot be submitted	Course staff

12.3 Assumptions

Assumption	Impact if incorrect	Owners
The Toolshop application remains stable during testing	Test results may be invalid; retesting required	QA Team
All testers have required tools installed and configured	Setup delays, reduced test coverage	Each tester
Localhost (Docker) environment is representative of production	Performance results may not match real-world behavior	QA Team
Deliverables in Markdown/PDF are acceptable for submission	Reformatting may be required at the last minute	QA Team Lead

12.4 Constraints

Constraint	Impact on Test Effort	Owners
Strict deadline	Limited time for multiple test cycles; forces risk-based testing	Entire QA Team
Environment limited to localhost (no staging/production)	Cannot validate in production-like conditions	QA Team
Hardware constraints (student laptops, not servers)	Performance test results approximate only	QA Team
Group project workload sharing	Requires close coordination; delays by one member impact all	Entire QA Team

Chapter 13

Management Process and Procedures

13.1 Measuring and Assessing the Extent of Testing

The extent of testing for *The Toolshop* will be measured by:

- Percentage of test cases executed vs. planned (target 85%).
- Pass/Fail ratio across functional, GUI, API, automation, and performance tests.
- Defect density (number of defects per module or per 100 test cases).
- Performance KPIs such as average response time and error rate from JMeter runs.

Assessment will occur at the end of each test cycle and be consolidated into the Final Test Report.

13.2 Assessing the Deliverables of this Test Plan

Deliverables (Test Plan, Test Cases, Test Case Summary, Bug Reports, Test Report) will be assessed based on:

- Completeness: coverage of all required testing types (HW#02–HW#07).
- Accuracy: consistency between test results and documented defects.
- Quality: clarity, structure, and compliance with course requirements.
- Timeliness: submitted before the official deadline.

Peer review within the team and final review by the Test Manager will ensure quality before submission.

13.3 Problem Reporting, Escalation, and Issue Resolution

- **Reporting**: All defects will be logged in GitHub Issues or a shared spreadsheet with severity (Critical, Major, Minor), status, and reproduction steps.
- **Escalation**: Blocker issues (e.g., build inaccessible, database crash) will be escalated immediately to the Test Manager.
- **Resolution**: Developers or environment owners will address issues; QA team retests to confirm closure.
- All escalated items will be documented in the Incident Log.

13.4 Managing Test Cycles

- Each cycle (manual, API, automation, performance) will start only after entry criteria are met (stable build, seeded DB).
- At the end of each cycle, the QA team will:
  - Summarize executed cases, defects, and coverage.
  - Conduct a bug triage session to prioritize fixes.
  - Decide whether to continue, repeat, or terminate the cycle.
- Regression cycle will run before final submission using smoke and automated tests.

13.5 Traceability Strategies

Traceability will be ensured by:

- Linking requirements (from assignment description) → Test Cases → Bugs.
- Maintaining a Traceability Matrix in Excel or Google Sheets.
- Using unique IDs (e.g., LOGIN-001 for test cases, BUG-001 for defects) to enable cross-referencing.
- Ensuring all high-priority requirements (login, checkout, payment) are covered by at least one test case.

13.6 Approval and Signoff

- Draft deliverables will be peer-reviewed within the QA team.
- The Test Manager (QA Team Lead) will provide internal signoff before submission.
- Final approval lies with the course instructor and TAs upon grading.
- Once signed off, the Test Plan and supporting deliverables will be considered baseline for HW#08 submission.